Chương 15

Hồi quy softmax

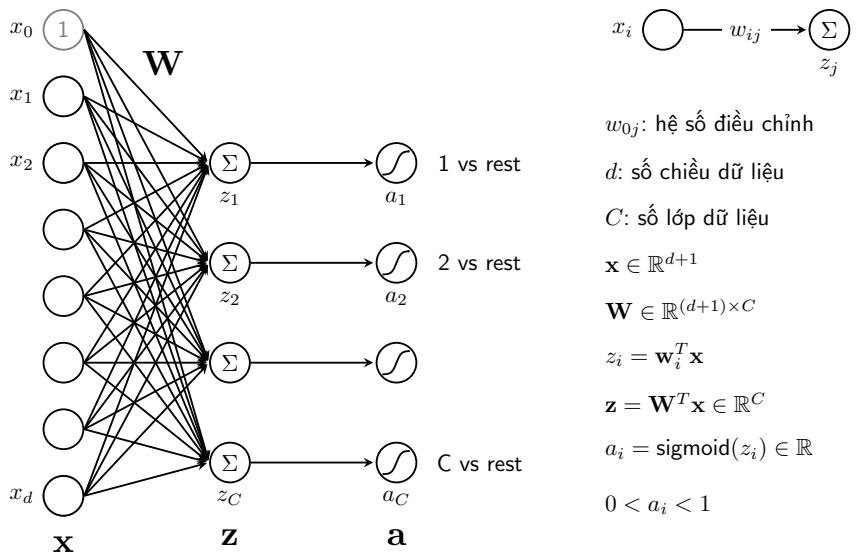
Các bài toán phân loại thực tế thường có nhiều lớp dữ liệu. Như đã thảo luận trong Chương 14, các bộ phân loại nhị phân tuy có thể kết hợp với nhau để giải quyết các bài toán phân loại đa lớp nhưng chúng vẫn có những hạn chế nhất định. Trong chương này, một phương pháp mở rộng của hồi quy logistic có tên là *hồi quy softmax* sẽ được giới thiệu nhằm khắc phục những hạn chế đã đề cập. Một lần nữa, mặc dù trong tên có chứa từ "hồi quy", hồi quy softmax được sử dụng cho các bài toán phân loại. Hồi quy softmax là một trong những thành phần phổ biến nhất trong các bộ phân loại hiện nay.

15.1. Giới thiệu

Với bài toán phân loại nhị phân sử dụng hồi quy logistic, đầu ra của mạng neural là một số thực trong khoảng $(0, 1)$, có ý nghĩa như xác suất để đầu vào thuộc một trong hai lớp. Ý tưởng này cũng có thể mở rộng cho bài toán phân loại đa lớp, ở đó có C nút ở tầng đầu ra và giá trị mỗi nút đóng vai trò như xác suất để đầu vào rơi vào lớp tương ứng. Như vậy, các đầu ra này liên kết với nhau qua việc chúng đều là các số dương và có tổng bằng một. Mô hình hồi quy softmax thảo luận trong chương này đảm bảo tính chất đó.

Nhắc lại biểu diễn dưới dạng mạng neural của kỹ thuật *one-vs-rest* như trong Hình 15.1. Tầng đầu ra có thể tách thành hai *tầng con* $\mathbf{z}$ và $\mathbf{a}$. Mỗi thành phần của tầng con thứ hai a_i chỉ phụ thuộc vào thành phần tương ứng ở tầng con thứ nhất z_i thông qua hàm sigmoid $a_i = \sigma(z_i)$. Các giá trị đầu ra a_i đều là các số dương nhưng vì không có ràng buộc giữa chúng, tổng các xác suất này không đảm bảo bằng một.

Các mô hình hồi quy tuyến tính, PLA, và hồi quy logistic chỉ có một nút ở tầng đầu ra. Trong các trường hợp đó, tham số mô hình chỉ là một vector $\mathbf{w}$.



Hình 15.1. Phân loại đa lớp với hồi quy logistic và one-vs-rest.

Trong trường hợp tầng đầu ra có nhiều hơn một nút, tham số mô hình sẽ là tập hợp tham số $\mathbf{w}_i$ ứng với từng nút. Lúc này, ta có một *ma trận trọng số* $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \ldots, \mathbf{w}_C]$, mỗi cột ứng với một nút ở tầng đầu ra.

15.2. Hàm softmax

15.2.1. Hàm softmax

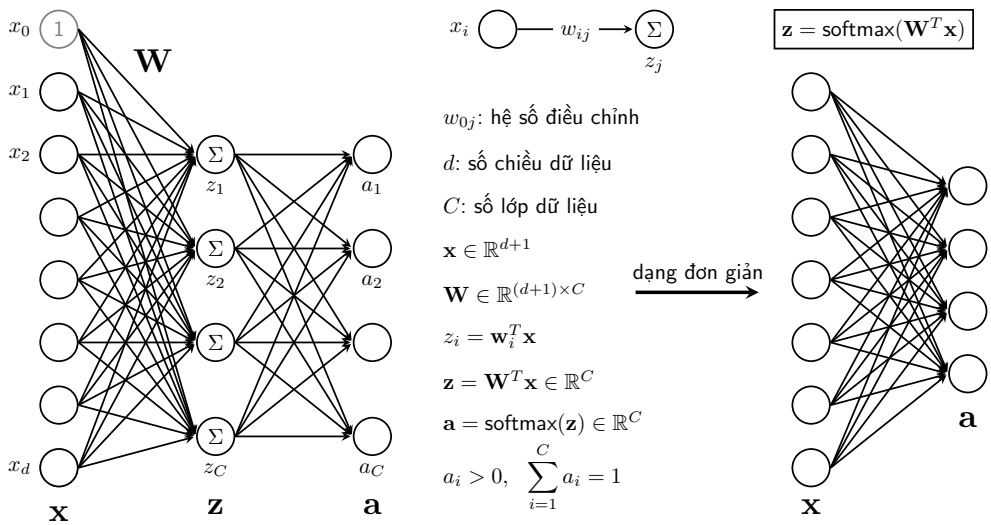
Chúng ta cần một mô hình xác suất sao cho với mỗi đầu vào $\mathbf{x}$, a_i thể hiện xác suất để đầu vào đó rơi vào lớp thứ i. Vậy điều kiện cần là các a_i phải dương và tổng của chúng bằng một. Ngoài ra, ta sẽ thêm điều kiện giá trị $z_i = \mathbf{x}^T\mathbf{w}_i$ càng lớn thì xác suất dữ liệu rơi vào lớp thứ i càng cao. Điều kiện cuối này chỉ ra rằng ta cần một quan hệ đồng biến.

Chú ý rằng z_i có thể nhận giá trị cả âm và dương vì nó là một tổ hợp tuyến tính các thành phần của vector đặc trưng $\mathbf{x}$. Một hàm số khả vi đồng biến đơn giản có thể biến z_i thành một giá trị dương là hàm $\exp(z_i) = e^{z_i}$. Hàm số này không những khả vi mà còn có đạo hàm bằng chính nó, việc này mang lại nhiều lợi ích khi tối ưu. Điều kiện tổng các a_i bằng một có thể được đảm bảo nếu

$$a_i = \frac{\exp(z_i)}{\sum_{j=1}^{C}\exp(z_j)}, \quad \forall i = 1, 2, \ldots, C. \tag{15.1}$$

Mối quan hệ này thoả mãn tất cả các điều kiện đã xét: các đầu ra a_i dương, có tổng bằng một và giữ được thứ tự của z_i. Hàm số này được gọi là *hàm softmax*. Lúc này, ta có thể coi rằng

$$p(y_k = i|\mathbf{x}_k; \mathbf{W}) = a_i \tag{15.2}$$



Hình 15.2. Mô hình hồi quy softmax dưới dạng neural network.

Trong đó, $p(y = i|\mathbf{x}; \mathbf{W})$ được hiểu là xác suất để một điểm dữ liệu $\mathbf{x}$ rơi vào lớp thứ i nếu biết tham số mô hình là ma trận trọng số $\mathbf{W}$. Hình 15.2 thể hiện mô hình hồi quy softmax dưới dạng mạng neural. Mô hình này khác one-vs-rest nằm ở chỗ nó có các liên kết giữa mọi nút của hai tầng con $\mathbf{z}$ và $\mathbf{a}$.

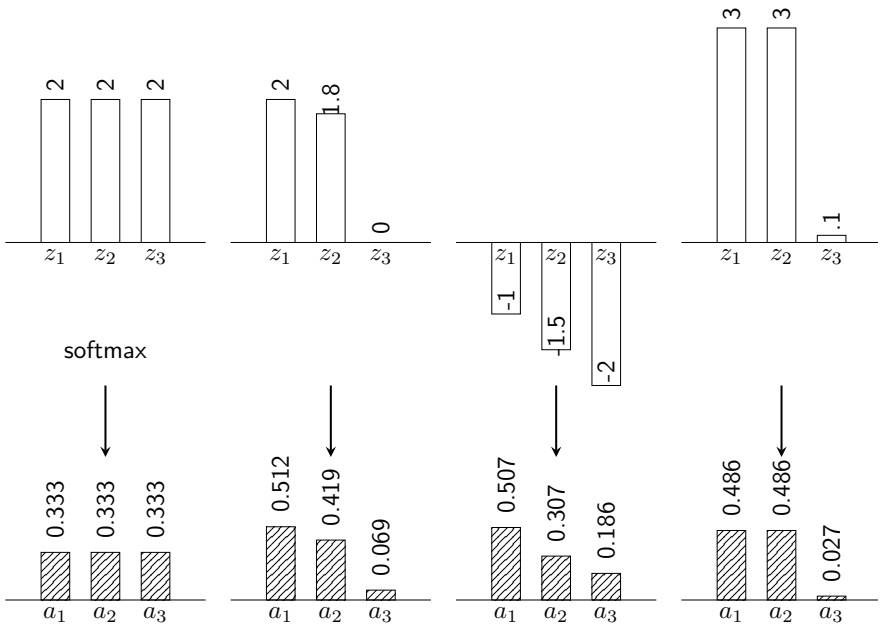
15.2.2. Xây dựng hàm softmax trong Python

Dưới đây là một đoạn code thực hiện hàm softmax. Đầu vào là một ma trận với mỗi *hàng* là một vector $\mathbf{z}$, đầu ra cũng là một ma trận mà mỗi hàng có giá trị là $\mathbf{a} = \text{softmax}(\mathbf{z})$. Các giá trị của $\mathbf{z}$ còn được gọi là *score*:

```
import numpy as np
def softmax(Z):
    """
    Compute softmax values for each sets of scores in Z.
    each column of Z is a set of scores.
    Z: a numpy array of shape (N, C)
    return a numpy array of shape (N, C)
    """
    e_Z = np.exp(Z)
    A = e_Z / e_Z.sum(axis = 1, keepdims = True)
    return A
```

15.2.3. Một vài ví dụ

Hình 15.3 mô tả một vài ví dụ về mối quan hệ giữa đầu vào và đầu ra của hàm softmax. Hàng trên thể hiện các score z_i với giả sử rằng số lớp dữ liệu là ba. Hàng dưới thể hiện các giá trị đầu ra a_i của hàm softmax.



Hình 15.3. Một số ví dụ về đầu vào và đầu ra của hàm softmax.

Có một vài quan sát như sau:

- Cột 1: Nếu các z_i bằng nhau (bằng 2 hoặc một số bất kỳ) thì các a_i cũng bằng nhau và bằng 1/3.
- Cột 2: Nếu giá trị lớn nhất trong các z_i là z_1 vẫn bằng 2, thì mặc dù xác suất tương ứng a_1 vẫn là lớn nhất, nó đã tăng lên hơn 0.5. Sự chênh lệch ở đầu ra là đáng kể, nhưng thứ tự tương ứng không thay đổi.
- Cột 3: Khi các giá trị z_i là âm thì các giá trị a_i vẫn là dương và thứ tự vẫn được đảm bảo.
- Cột 4: Nếu $z_1 = z_2$ thì $a_1 = a_2$.

Bạn đọc có thể thử với các giá trị khác trên trình duyệt tại https://goo.gl/pKxQYc, phần softmax.

15.2.4. Phiên bản ổn định hơn của hàm softmax

Khi một trong các z_i quá lớn, việc tính toán $\exp(z_i)$ có thể gây ra hiện tượng tràn số, ảnh hưởng lớn tới kết quả của hàm softmax. Có một cách khắc phục hiện tượng này dựa trên quan sát

$$\frac{\exp(z_i)}{\sum_{j=1}^{C}\exp(z_j)} = \frac{\exp(-c)\exp(z_i)}{\exp(-c)\sum_{j=1}^{C}\exp(z_j)} = \frac{\exp(z_i - c)}{\sum_{j=1}^{C}\exp(z_j - c)} \tag{15.3}$$

với c là một hằng số bất kỳ. Từ đây, một kỹ thuật đơn giản giúp khắc phục hiện tượng tràn số là trừ tất cả các z_i đi một giá trị đủ lớn. Trong thực nghiệm, giá trị đủ lớn này thường được chọn là $c = \max z_i$. Ta có thể cải tiến đoạn code cho hàm `softmax` phía trên bằng cách trừ mỗi hàng của ma trận đầu vào `Z` đi giá trị lớn nhất trong hàng đó. Ta có phiên bản ổn định hơn là `softmax_stable`[40]:

```
def softmax_stable(Z):
    """
    Compute softmax values for each set of scores in Z.
    each row of Z is a set of scores.
    """
    e_Z = np.exp(Z - np.max(Z, axis = 1, keepdims = True))
    A = e_Z / e_Z.sum(axis = 1, keepdims = True)
    return A
```

15.3. Hàm mất mát và phương pháp tối ưu

15.3.1. Entropy chéo

Đầu ra của mạng softmax, $\mathbf{a} = \text{softmax}(\mathbf{W}^T\mathbf{x})$, là một vector có số phần tử bằng số lớp dữ liệu. Các phần tử của vector này là các số dương có tổng bằng một, thể hiện xác suất để điểm đầu vào rơi vào từng lớp dữ liệu. Với một điểm dữ liệu huấn luyện thuộc lớp thứ c, chúng ta mong muốn xác suất tương ứng với lớp này càng cao càng tốt, tức càng gần một càng tốt. Việc này kéo theo các phần tử còn lại gần với không. Một cách tự nhiên, đầu ra thực sự $\mathbf{y}$ là một vector có tất cả các phần tử bằng không trừ phần từ ở vị trí thứ c bằng một. Cách biểu diễn nhãn dưới dạng vector này được gọi là mã hoá *one-hot*.

Hàm mất mát của hồi quy softmax được xây dựng dựa trên bài toán tối thiểu sự khác nhau giữa đầu ra dự đoán $\mathbf{a}$ và đầu ra thực sự $\mathbf{y}$ ở dạng one-hot. Khi cả hai là các vector thể hiện xác suất, khoảng cách giữa chúng thường được đo bằng một hàm số được gọi là *entropy chéo* $H(\mathbf{y}, \mathbf{a})$. Đặc điểm nổi bật của hàm số

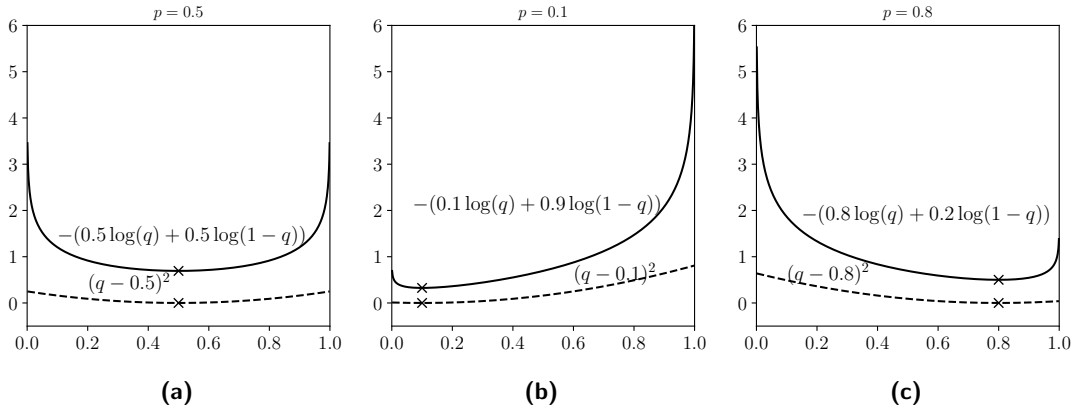
Một đặc điểm nổi bật là nếu cố định $\mathbf{y}$, hàm số sẽ đạt giá trị nhỏ nhất khi $\mathbf{a} = \mathbf{y}$, và càng lớn nếu $\mathbf{a}$ càng khác $\mathbf{y}$.

Entropy chéo giữa hai vector phân phối $\mathbf{p}$ và $\mathbf{q}$ rời rạc được định nghĩa bởi

$$H(\mathbf{p}, \mathbf{q}) = -\sum_{i=1}^{C} p_i \log q_i \tag{15.4}$$

Hình 15.4 thể hiện ưu điểm của hàm entropy chéo so với hàm bình phương khoảng cách Euclid. Đây là ví dụ trong trường hợp $C = 2$ và p_1 lần lượt nhận các giá trị $0.5, 0.1$ và 0.8 và $p_2 = 1 - p_1$. Có hai nhận xét quan trọng:

[40] Xem thêm về cách xử lý mảng numpy trong Python tại https://fundaml.com



Hình 15.4. So sánh hàm entropy chéo (đường nét liền) và hàm bình phương khoảng cách (đường nét đứt). Các điểm được đánh dấu thể hiện điểm cực tiểu toàn cục của mỗi hàm. Càng xa điểm cực tiểu toàn cục, khoảng cách giữa hai hàm số càng lớn.

- Giá trị nhỏ nhất của cả hai hàm số đạt được khi $q = p$ tại hoành độ các điểm được đánh dấu.
- Nhận thấy rằng hàm entropy chéo nhận giá trị rất cao, tức mất mát rất cao, khi q ở xa p. Sự chênh lệch giữa các mất mát ở gần hay xa nghiệm của hàm bình phương khoảng cách $(q-p)^2$ là ít đáng kể hơn. Về mặt tối ưu, hàm entropy chéo sẽ cho nghiệm gần với p hơn vì những nghiệm ở xa gây ra mất mát lớn.

Hai tính chất trên đây khiến hàm entropy chéo được sử dụng rộng rãi khi tính khoảng cách giữa hai phân phối xác suất. Tiếp theo, chúng ta sẽ chứng minh nhận định sau.

Cho $\mathbf{p} \in \mathbb{R}_+^C$ *là một vector với các thành phần dương có tổng bằng một. Bài toán tối ưu*

$$\mathbf{q} = \arg\min_{\mathbf{q}} H(\mathbf{p}, \mathbf{q})$$

$$\text{thoả mãn: } \sum_{i=1}^{C} q_i = 1; q_i > 0$$

có nghiệm $\mathbf{q} = \mathbf{p}$.

Bài toán này có thể giải quyết bằng phương pháp nhân tử Lagrange (xem Phụ lục A).

Lagrangian của bài toán tối ưu này là

$$\mathcal{L}(q_1, q_2, \ldots, q_C, \lambda) = -\sum_{i=1}^{C} p_i \log(q_i) + \lambda(\sum_{i=1}^{C} q_i - 1)$$

Ta cần giải hệ phương trình

$$\nabla_{q_1,\ldots,q_C,\lambda}\mathcal{L}(q_1,\ldots,q_C,\lambda) = 0 \Leftrightarrow \begin{cases} -\frac{p_i}{q_i} + \lambda = 0, \quad i = 1,\ldots,C \\ q_1 + q_2 + \cdots + q_C = 1 \end{cases}$$

Từ phương trình thứ nhất ta có $p_i = \lambda q_i$. Vì vậy, $1 = \sum_{i=1}^{C} p_i = \lambda \sum_{i=1}^{C} q_i = \lambda \Rightarrow \lambda = 1$. Điều này tương đương với $q_i = p_i, \forall i$.

Chú ý

a. *Hàm entropy chéo không có tính đối xứng $H(\mathbf{p},\mathbf{q}) \neq H(\mathbf{q},\mathbf{p})$. Điều này có thể nhận ra từ việc các thành phần của $\mathbf{p}$ trong công thức (15.4) có thể nhận giá trị bằng không, trong khi các thành phần của $\mathbf{q}$ phải là dương vì $\log(0)$ không xác định. Chính vì vậy, khi sử dụng entropy chéo trong các bài toán phân loại, $\mathbf{p}$ là đầu ra thực sự ở dạng one-hot, $\mathbf{q}$ là đầu ra dự đoán. Trong các thành phần thể hiện xác suất của $\mathbf{q}$, không có thành phần nào tuyệt đối bằng một hoặc tuyệt đối bằng không do hàm* exp *luôn trả về một giá trị dương.*

b. *Khi $\mathbf{p}$ là một vector ở dạng one-hot, giả sử chỉ có $p_c = 1$, biểu thức entropy chéo trở thành $-\log(q_c)$. Biểu thức này đạt giá trị nhỏ nhất nếu $q_c = 1$, điều này không xảy ra vì nghiệm này không thuộc miền xác định của bài toán. Tuy nhiên, giá trị entropy chéo tiệm cận tới không khi q_c tiến đến một, tức z_c rất rất lớn so với các z_i còn lại.*

15.3.2. Xây dựng hàm mất mát

Trong trường hợp có C lớp dữ liệu, mất mát giữa đầu ra dự đoán và đầu ra thực sự của một điểm dữ liệu $\mathbf{x}_i$ với nhãn $\mathbf{y}_i$ được tính bởi

$$J_i(\mathbf{W}) \triangleq J(\mathbf{W};\mathbf{x}_i,\mathbf{y}_i) = -\sum_{j=1}^{C} y_{ji}\log(a_{ji}) \tag{15.5}$$

với y_{ji} và a_{ji} lần lượt là phần tử thứ j của vector xác suất $\mathbf{y}_i$ và $\mathbf{a}_i$. Nhắc lại rằng đầu ra $\mathbf{a}_i$ phụ thuộc vào đầu vào $\mathbf{x}_i$ và ma trận trọng số $\mathbf{W}$. Tới đây, nếu để ý rằng chỉ có đúng một j sao cho $y_{ji} = 1, \forall i$, biểu thức (15.5) chỉ còn lại một số hạng tương ứng với giá trị j đó. Để tránh việc sử dụng quá nhiều ký hiệu, chúng ta giả sử rằng y_i là nhãn của điểm dữ liệu $\mathbf{x}_i$ (các nhãn là các số tự nhiên từ 1 tới C), khi đó j chính bằng y_i. Sau khi có ký hiệu này, ta có thể viết lại

$$J_i(\mathbf{W}) = -\log(a_{y_i,i}) \tag{15.6}$$

với $a_{y_i,i}$ là phần tử thứ y_i của vector $\mathbf{a}_i$.

Khi sử dụng toàn bộ tập huấn luyện $\mathbf{x}_i, \mathbf{y}_i, i = 1,2,\ldots,N$, hàm mất mát của hồi quy softmax được xác định bởi

$$J(\mathbf{W}; \mathbf{X}, \mathbf{Y}) = -\frac{1}{N}\sum_{i=1}^{N}\log(a_{y_i,i}) \tag{15.7}$$

Ở đây, ma trận trọng số $\mathbf{W}$ là biến cần tối ưu. Hàm mất mát này có gradient khá gọn, kỹ thuật tính gradient gần giống với hồi quy logistic. Để tránh quá khớp, ta cũng có thể sử dụng cơ chế kiểm soát suy giảm trọng số:

$$\bar{J}(\mathbf{W}; \mathbf{X}, \mathbf{Y}) = -\frac{1}{N}\left(\sum_{i=1}^{N}\log(a_{y_i,i}) + \frac{\lambda}{2}\|\mathbf{W}\|_F^2\right) \tag{15.8}$$

Trong các mục tiếp theo, chúng ta sẽ làm việc với hàm mất mát (15.7). Việc mở rộng cho hàm mất mát với cơ chế kiểm soát (15.8) không phức tạp vì gradient của số hạng kiểm soát $\frac{\lambda}{2}\|\mathbf{W}\|_F^2$ đơn giản là $\lambda\mathbf{W}$. Hàm mất mát (15.7) có thể được thực hiện trên Python như sau[41]:

```
def softmax_loss(X, y, W):
    """
    W: 2d numpy array of shape (d, C),
    each column correspoding to one output node
    X: 2d numpy array of shape (N, d), each row is one data point
    y: 1d numpy array  --  label of each row of X
    """
    A = softmax_stable(X.dot(W))
    id0 = range(X.shape[0]) # indexes in axis 0, indexes in axis 1 are in y
    return -np.mean(np.log(A[id0, y]))
```

Chú ý

a. *Khi biểu diễn dưới dạng toán học, mỗi điểm dữ liệu là một cột của ma trận* $\mathbf{X}$*; nhưng khi làm việc với numpy, mỗi điểm dữ liệu được đọc theo* `axis = 0` *của mảng hai chiều* `X`*. Việc này thống nhất với các thư viện scikit-learn hay tensorflow ở việc* `X[i]` *được dùng để chỉ điểm dữ liệu thứ* `i`*, tính từ* `0`*. Tức là, nếu có* N *điểm dữ liệu trong không gian* d *chiều thì* $\mathbf{X} \in \mathbb{R}^{d\times N}$*, nhưng* `X.shape == (N, d)`.

b. $\mathbf{W} \in \mathbb{R}^{d\times C}$, `W.shape == (d, C)`.

c. $\mathbf{W}^T\mathbf{X}$ *sẽ được biểu diễn bởi* `X.dot(W)`*, và có* `shape == (N, C)`.

d. *Khi làm việc với phép nhân ma trận hay mảng nhiều chiều trong numpy, cần chú ý tới kích thước của các ma trận sao cho các phép nhân thực hiện được.*

15.3.3. Tối ưu hàm mất mát

Hàm mất mát sẽ được tối ưu bằng gradient descent, cụ thể là mini-batch gradient descent. Mỗi lần cập nhật của mini-batch gradient descent được thực hiện trên

[41] Truy cập vào nhiều phần tử của mảng hai chiều trong numpy - FundaML https://goo.gl/SzLDxa.

một batch có số phần tử $1 < k \ll N$. Để tính được gradient của hàm mất mát theo tập con này, trước hết ta xem xét gradient của hàm mất mát tại một điểm dữ liệu.

Với chỉ một cặp dữ liệu $(\mathbf{x}_i, \mathbf{y}_i)$, ta dùng (15.5)

$$\begin{aligned} J_i(\mathbf{W}) = -\sum_{j=1}^{C} y_{ji}\log(a_{ji}) &= -\sum_{j=1}^{C} y_{ji}\log\left(\frac{\exp(\mathbf{x}_i^T\mathbf{w}_j)}{\sum_{k=1}^{C}\exp(\mathbf{x}_i^T\mathbf{w}_k)}\right) \\ &= -\sum_{j=1}^{C}\left(y_{ji}\mathbf{x}_i^T\mathbf{w}_j - y_{ji}\log\left(\sum_{k=1}^{C}\exp(\mathbf{x}_i^T\mathbf{w}_k)\right)\right) \\ &= -\sum_{j=1}^{C} y_{ji}\mathbf{x}_i^T\mathbf{w}_j + \log\left(\sum_{k=1}^{C}\exp(\mathbf{x}_i^T\mathbf{w}_k)\right) \end{aligned} \tag{15.9}$$

Tiếp theo ta sử dụng công thức

$$\nabla_{\mathbf{W}} J_i(\mathbf{W}) = \left[\nabla_{\mathbf{w}_1} J_i(\mathbf{W}), \nabla_{\mathbf{w}_2} J_i(\mathbf{W}), \ldots, \nabla_{\mathbf{w}_C} J_i(\mathbf{W})\right]. \tag{15.10}$$

Trong đó, gradient theo từng cột của $\mathbf{w}_j$ có thể tính được dựa theo (15.9) và quy tắc chuỗi:

$$\begin{aligned} \nabla_{\mathbf{w}_j} J_i(\mathbf{W}) &= -y_{ji}\mathbf{x}_i + \frac{\exp(\mathbf{x}_i^T\mathbf{w}_j)}{\sum_{k=1}^{C}\exp(\mathbf{x}_i^T\mathbf{w}_k)}\mathbf{x}_i \\ &= -y_{ji}\mathbf{x}_i + a_{ji}\mathbf{x}_i = \mathbf{x}_i(a_{ji} - y_{ji}) \\ &= e_{ji}\mathbf{x}_i \text{ (với } e_{ji} = a_{ji} - y_{ji}) \end{aligned} \tag{15.11}$$

Giá trị $e_{ji} = a_{ji} - y_{ji}$ chính là sự sai khác giữa đầu ra dự đoán và đầu ra thực sự tại thành phần thứ j. Kết hợp (15.10) và (15.11) với $\mathbf{e}_i = \mathbf{a}_i - \mathbf{y}_i$, ta có

$$\nabla_{\mathbf{W}} J_i(\mathbf{W}) = \mathbf{x}_i[e_{1i}, e_{2i}, \ldots, e_{Ci}] = \mathbf{x}_i\mathbf{e}_i^T \tag{15.12}$$

$$\Rightarrow \nabla_{\mathbf{W}} J(\mathbf{W}) = \frac{1}{N}\sum_{i=1}^{N}\mathbf{x}_i\mathbf{e}_i^T = \frac{1}{N}\mathbf{X}\mathbf{E}^T \tag{15.13}$$

với $\mathbf{E} = \mathbf{A} - \mathbf{Y}$. Công thức đơn giản này giúp cả batch gradient descent và mini-batch gradient descent có thể dễ dàng được áp dụng. Trong trường hợp mini-batch gradient, giả sử kích thước batch là k, ký hiệu $\mathbf{X}_b \in \mathbb{R}^{d\times k}, \mathbf{Y}_b \in \{0,1\}^{C\times k}, \mathbf{A}_b \in \mathbb{R}^{C\times k}$ là dữ liệu ứng với một batch, công thức cập nhật cho một batch sẽ là

$$\mathbf{W} \leftarrow \mathbf{W} - \frac{\eta}{N_b}\mathbf{X}_b\mathbf{E}_b^T \tag{15.14}$$

với N_b là kích thước của mỗi batch và η là tốc độ học.

Hàm số tính gradient theo $\mathbf{W}$ trong Python có thể được thực hiện như sau:

```
def softmax_grad(X, y, W):
    """
    W: 2d numpy array of shape (d, C),
    each column correspoding to one output node
    X: 2d numpy array of shape (N, d), each row is one data point
    y: 1d numpy array  --  label of each row of X
    """
    A = softmax_stable(X.dot(W)) # shape of (N, C)
    id0 = range(X.shape[0])
    A[id0, y] -= 1  # A - Y, shape of (N, C)
    return X.T.dot(A)/X.shape[0]
```

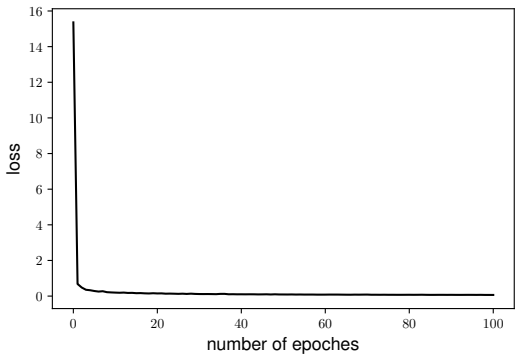
Bạn đọc có thể kiểm tra lại sự chính xác của việc tính gradient này bằng hàm `check_grad`.

Từ đó, ta có thể viết hàm số huấn luyện hồi quy softmax như sau:

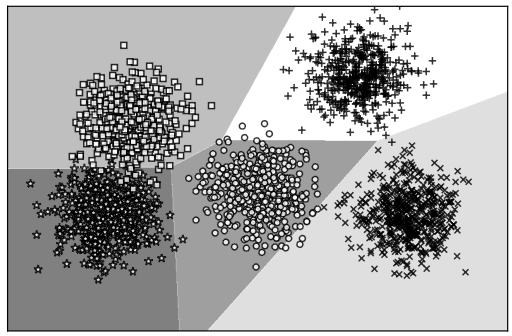
```
def softmax_fit(X, y, W, lr = 0.01, nepochs = 100, tol = 1e-5, batch_size =
    10):
    W_old = W.copy()
    ep = 0
    loss_hist = [loss(X, y, W)] # store history of loss
    N = X.shape[0]
    nbatches = int(np.ceil(float(N)/batch_size))
    while ep < nepochs:
        ep += 1
        mix_ids = np.random.permutation(N) # stochastic
        for i in range(nbatches):
            # get the i-th batch
            batch_ids = mix_ids[batch_size*i:min(batch_size*(i+1), N)]
            X_batch, y_batch = X[batch_ids], y[batch_ids]
            W -= lr*softmax_grad(X_batch, y_batch, W) # gradient descent
            loss_hist.append(softmax_loss(X, y, W))
        if np.linalg.norm(W - W_old)/W.size < tol:
            break
        W_old = W.copy()
    return W, loss_hist
```

Cuối cùng là hàm dự đoán nhãn của các điểm dữ liệu mới. Nhãn của một điểm dữ liệu mới được xác định bằng chỉ số của lớp dữ liệu có xác suất rơi vào cao nhất, và cũng chính là chỉ số của score cao nhất.

```
def pred(W, X):
    """
    predict output of each columns of X . Class of each x_i is determined by
    location of the max probability. Note that classes are indexed from 0.
    """
    return np.argmax(X.dot(W), axis =1)
```



(a)



(b)

Hình 15.5. Ví dụ về sử dụng hồi quy softmax cho năm lớp dữ liệu. (a) Giá trị hàm mất mát qua các epoch. (b) Kết quả phân loại cuối cùng.

15.4. Ví dụ trên Python

Để minh hoạ ranh giới của các lớp dữ liệu khi sử dụng hồi quy softmax, chúng ta cùng làm một ví dụ nhỏ trong không gian hai chiều với năm lớp dữ liệu:

```
C, N = 5, 500    # number of classes and number of points per class
means = [[2, 2], [8, 3], [3, 6], [14, 2], [12, 8]]
cov = [[1, 0], [0, 1]]
X0 = np.random.multivariate_normal(means[0], cov, N)
X1 = np.random.multivariate_normal(means[1], cov, N)
X2 = np.random.multivariate_normal(means[2], cov, N)
X3 = np.random.multivariate_normal(means[3], cov, N)
X4 = np.random.multivariate_normal(means[4], cov, N)
X = np.concatenate((X0, X1, X2, X3, X4), axis = 0) # each row is a datapoint
Xbar = np.concatenate((X, np.ones((X.shape[0], 1))), axis = 1) # bias trick

y = np.asarray([0]*N + [1]*N + [2]*N+ [3]*N + [4]*N) # label
W_init = np.random.randn(Xbar.shape[1], C)
W, loss_hist = softmax_fit(Xbar, y, W_init, lr = 0.05)
```

Giá trị của hàm mất mát qua các epoch được cho trên Hình 15.5a. Ta thấy rằng hàm mất mát giảm rất nhanh sau đó hội tụ. Các điểm dữ liệu huấn luyện của mỗi lớp là các điểm có hình dạng khác nhau trong Hình 15.5b. Các phần có nền khác nhau thể hiện vùng của mỗi lớp dữ liệu tìm được bằng hồi quy softmax. Ta thấy rằng các đường ranh giới có dạng đường thẳng. Kết quả phân chia vùng cũng khá tốt, chỉ có một số ít điểm trong tập huấn luyện bị phân loại sai. Ta cũng thấy hồi quy softmax tốt hơn rất nhiều so với phương pháp kết hợp các bộ phân loại nhị phân.

MNIST với hồi quy softmax trong scikit-learn

Trong scikit-learn, hồi quy softmax được tích hợp trong class `sklearn.linear_model.LogisticRegression`. Như sẽ thấy trong phần thảo luận, hồi quy logistic chính là hồi quy softmax cho bài toán phân loại nhị phân. Với bài toán phân loại đa lớp, thư viện này mặc định sử dụng kỹ thuật one-vs-rest. Để sử dụng hồi quy softmax, ta thay đổi thuộc tính `multi_class = 'multinomial'` và `solver = 'lbfgs'`. Ở đây, `'lbfgs'` là một phương pháp tối ưu rất mạnh cũng dựa trên gradient. Trong khuôn khổ của cuốn sách, chúng ta sẽ không thảo luận về phương pháp này[42].

Quay lại với bài toán phân loại chữ số viết tay trong cơ sở dữ liệu MNIST. Đoạn code dưới đây thực hiện việc lấy ra 10000 điểm dữ liệu trong số 70000 điểm làm tập kiểm tra, còn lại là tập huấn luyện. Bộ phân loại được sử dụng là hồi quy softmax.

```
import numpy as np
from sklearn.datasets import fetch_mldata
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
mnist = fetch_mldata('MNIST original', data_home='../../data/')

X = mnist.data
y = mnist.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=10000)

model = LogisticRegression(C = 1e5,
solver = 'lbfgs', multi_class = 'multinomial') # C is inverse of lam
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy %.2f %%" % (100*accuracy_score(y_test, y_pred.tolist())))
```

Kết quả:

```
Accuracy: 92.19 %
```

So với kết quả hơn 91.7% của one-vs-rest hồi quy logistic, kết quả của hồi quy softmax đã được cải thiện. Kết quả thấp này hoàn toàn có thể dự đoán được vì thực ra hồi quy softmax chỉ tạo ra các đường ranh giới tuyến tính. Kết quả tốt nhất của bài toán phân loại chữ số trong MNIST hiện nay vào khoảng hơn 99.7%, đạt được bằng một mạng neuron tích chập với rất nhiều tầng ẩn và tầng cuối cùng là một hồi quy softmax.

[42] Đọc thêm: *Limited-memory BFGS – Wikipedia* (https://goo.gl/qf1kmn).

15.5. Thảo luận

15.5.1. Hồi quy logistic là trường hợp đặc biệt của hồi quy softmax

Khi $C = 2$, hồi quy softmax và hồi quy logistic là hai mô hình giống nhau. Thật vậy, với $C = 2$, đầu ra của hàm softmax cho một đầu vào $\mathbf{x}$ là:

$$a_1 = \frac{\exp(\mathbf{x}^T\mathbf{w}_1)}{\exp(\mathbf{x}^T\mathbf{w}_1) + \exp(\mathbf{x}^T\mathbf{w}_2)} = \frac{1}{1 + \exp(\mathbf{x}^T(\mathbf{w}_2 - \mathbf{w}_1))}; \quad a_2 = 1 - a_1 \quad (15.15)$$

Từ đây ta thấy rằng, a_1 có dạng là một hàm sigmoid với vector trọng số có dạng $\mathbf{w} = -(\mathbf{w}_2 - \mathbf{w}_1)$. Khi $C = 2$, bạn đọc cũng có thể thấy rằng hàm mất mát của hồi quy logistic và hồi quy softmax là như nhau. Hơn nữa, mặc dù có hai đầu ra, hồi quy softmax có thể biểu diễn bởi một đầu ra vì tổng của chúng bằng một.

Giống như hồi quy logistic, hồi quy softmax được sử dụng trong các bài toán phân loại. Các tên gọi này được giữ lại vì vấn đề lịch sử.

15.5.2. Ranh giới tạo bởi hồi quy softmax là các mặt tuyến tính

Thật vậy, dựa vào hàm softmax thì một điểm dữ liệu $\mathbf{x}$ được dự đoán là rơi vào class j nếu $a_j \geq a_k, \ \forall k \neq j$. Bạn đọc có thể chứng minh được rằng:

$$a_j \geq a_k \Leftrightarrow z_j \geq z_k \Leftrightarrow \mathbf{x}^T\mathbf{w}_j \geq \mathbf{x}^T\mathbf{w}_k \ \Leftrightarrow \mathbf{x}^T(\mathbf{w}_j - \mathbf{w}_k) \geq 0. \quad (15.16)$$

Như vậy, một điểm thuộc lớp thứ j nếu và chỉ nếu $\mathbf{x}^T(\mathbf{w}_j - \mathbf{w}_k) \geq 0, \ \forall k \neq j$. Như vậy, mỗi lớp dữ liệu chiếm một vùng là giao của các nửa không gian. Nói cách khác, đường ranh giới giữa các lớp là các mặt tuyến tính.

15.5.3. Hồi quy softmax là một trong hai bộ phân loại phổ biến nhất

Hồi quy softmax cùng với máy vector hỗ trợ đa lớp (Chương 29) là hai bộ phân loại phổ biến nhất được dùng hiện nay. Hồi quy softmax đặc biệt được sử dụng nhiều trong các mạng neuron sâu với rất nhiều tầng ẩn. Những tầng phía trước có thể được coi như một bộ trích chọn vector đặc trưng, tầng cuối cùng thường là một hồi quy softmax.

Mã nguồn của chương này có thể được tìm thấy tại https://goo.gl/XU8ZXm.